TOPIC-3

Q1. Find Maximum and Minimum in an Array

Aim:

To find the maximum and minimum values in a given array.

Algorithm:

Step 1: Create an array of numbers.

Step 2: Assume the first element is both minimum and maximum.

Step 3: Traverse all elements in the array.

Step 4: Compare each element with the current minimum.

Step 5: Compare each element with the current maximum.

Step 6: Update the minimum and maximum values when necessary.

Step 7: Print the minimum and maximum values.

Python Code:

main.py

```
a = [5, 7, 3, 4, 9, 12, 6, 2]
minimum = min(a)
maximum = max(a)
print("Min =", minimum)
print("Max =", maximum)

```

Output:

Output

```
Min = 2
Max = 12

=== Code Execution
```

Time Complexity – O(N) | Space Complexity – O(1)

Q2. Find Maximum and Minimum in a Sorted Array

Aim:

To find the minimum and maximum values in an ascending sorted array.

Algorithm:

Step 1: Create a sorted array.

Step 2: The first element is the minimum value.

Step 3: The last element is the maximum value.

Step 4: Print the minimum and maximum values.

Python Code:

main.py

```
a = [2, 4, 6, 8, 10, 12, 14, 18]
minimum = a[0]
maximum = a[-1]
print("Min =", minimum)
print("Max =", maximum)

```

Output:

Output

```
Min = 2
Max = 18
```

Time Complexity – O(1) | Space Complexity – O(1)

Q3. Merge Sort

Aim:

To sort an unsorted array using the Merge Sort algorithm.

Algorithm:

Step 1: Divide the array into two halves.

Step 2: Recursively divide each half until one element remains.

Step 3: Compare elements from both halves.

Step 4: Merge the elements in sorted order.

Step 5: Repeat until the complete array is sorted.

Python Code:

main.py Visualize Run

```
def merge_sort(a):
    if len(a) > 1:
        mid = len(a) // 2
        left = a[:mid]
        right = a[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            a[k] = right[j]
            j += 1
            k += 1
a = [31, 23, 35, 27, 11, 21, 15, 28]
merge_sort(a)
print(a)
```

Output:

Output

```
[11, 15, 21, 23, 27, 28, 31, 35]

=== Code Execution Successful ===
```

Time Complexity – O(N log N) | Space Complexity – O(N)

Q4. Merge Sort with Comparison Count

Aim:

To implement Merge Sort and count the number of comparisons made during sorting.

Algorithm:

Step 1: Divide the array into smaller halves.

Step 2: Continue dividing until individual elements remain.

Step 3: Merge the smaller arrays.

Step 4: Compare elements while merging.

Step 5: Increase the comparison counter for every comparison.

Step 6: Print the sorted array and total comparisons.

Python Code:

main.py Visualize Run

```
comparisons = 0
def merge_sort(a):
    global comparisons
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        comparisons += 1
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
a = [12, 4, 78, 23, 45, 67, 89, 1]
sorted_array = merge_sort(a)
print("Sorted Array:", sorted_array)
print("Comparisons:", comparisons)
```

Output:

Output

```
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
```

Time Complexity – O(N log N) | Space Complexity – O(N)

Q5. Quick Sort Using First Element as Pivot

Aim:

To sort an array using Quick Sort by choosing the first element as the pivot.

Algorithm:

Step 1: Select the first element as the pivot.

Step 2: Divide the remaining elements into smaller and greater elements.

Step 3: Recursively apply Quick Sort to both sub-arrays.

Step 4: Combine the sorted left array, pivot, and sorted right array.

Step 5: Print the sorted array.

Python Code:

main.py Visualize Run

```
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[0]
    left = [x for x in a[1:] if x <= pivot]
    right = [x for x in a[1:] if x > pivot]
    print("Pivot:", pivot)
    print("Partition:", left + [pivot] + right)
    return quick_sort(left) + [pivot] + quick_sort(right)
a = [10, 16, 8, 12, 15, 6, 3, 9, 5]
result = quick_sort(a)
print("Sorted Array:", result)
```

Output:

```
Output

Pivot: 10
Partition: [8, 6, 3, 9, 5, 10, 16, 12, 15]
Pivot: 8
Partition: [6, 3, 5, 8, 9]
Pivot: 6
Partition: [3, 5, 6]
Pivot: 3
Partition: [3, 5]
Pivot: 16
Partition: [12, 15, 16]
Pivot: 12
Partition: [12, 15]
Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16]
```

Time Complexity – O(N log N) average | Space Complexity – O(N)

Q6. Quick Sort Using Middle Element as Pivot

Aim:

To sort an array using Quick Sort by selecting the middle element as the pivot.

Algorithm:

Step 1: Select the middle element as the pivot.

Step 2: Divide elements into smaller and greater groups.

Step 3: Recursively sort both groups.

Step 4: Combine the sorted arrays.

Step 5: Print the final sorted array.

Python Code:

main.py Visualize

```python
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[len(a) // 2]
    left = [x for x in a if x < pivot]
    middle = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]
    print("Pivot:", pivot)
    print("Partition:", left + middle + right)
    return quick_sort(left) + middle + quick_sort(right)
a = [19, 72, 35, 46, 58, 91, 22, 31]
result = quick_sort(a)
print("Sorted Array:", result)
```

Output:

Output

```
Pivot: 58
Partition: [19, 35, 46, 22, 31, 58, 72, 91]
Pivot: 46
Partition: [19, 35, 22, 31, 46]
Pivot: 22
Partition: [19, 22, 35, 31]
Pivot: 31
Partition: [31, 35]
Pivot: 91
Partition: [72, 91]
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]
```

Time Complexity – O(N log N) average | Space Complexity – O(N)

Q7. Binary Search with Comparison Count

Aim:

To search for an element using Binary Search and count the comparisons.

Algorithm:

Step 1: Set the first and last indexes.

Step 2: Find the middle index.

Step 3: Compare the middle element with the search key.

Step 4: If equal, return the position.

Step 5: If the key is greater, search the right half.

Step 6: Otherwise, search the left half.

Step 7: Count the comparisons.

Python Code:

main.py Visualize

```
a = [5, 10, 15, 20, 25, 30, 35, 40, 45]
key = 20
low = 0
high = len(a) - 1
comparisons = 0
while low <= high:
    mid = (low + high) // 2
    comparisons += 1
    if a[mid] == key:
        print("Position =", mid + 1)
        print("Comparisons =", comparisons)
        break
    elif a[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
```

Output:

Output

```
Position = 4
Comparisons = 4
```

Time Complexity – O(log N) | Space Complexity – O(1)

Q8. Binary Search with Mid-Point Steps

Aim:

To find an element using Binary Search and display the mid-point calculations.

Algorithm:

Step 1: Start with low = 0 and high = N-1.

Step 2: Calculate the middle index.

Step 3: Compare the middle element with the key.

Step 4: Move left or right depending on the comparison.

Step 5: Repeat until the element is found.

Python Code:

main.py Visualize

```
a = [3, 9, 14, 19, 25, 31, 42, 47, 53]
key = 31
low = 0
high = len(a) - 1
while low <= high:
    mid = (low + high) // 2
    print("Low =", low, "High =", high,
          "Mid =", mid, "Value =", a[mid])
    if a[mid] == key:
        print("Position =", mid + 1)
        break
    elif a[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
```

Output:

Output

```
Low = 0 High = 8 Mid = 4 Value = 25
Low = 5 High = 8 Mid = 6 Value = 42
Low = 5 High = 5 Mid = 5 Value = 31
Position = 6
```

Time Complexity – O(log N) | Space Complexity – O(1)

Q9. Find K Closest Points to Origin

Aim:

To find the K closest points to the origin using distance calculation.

Algorithm:

Step 1: Calculate the distance of every point from the origin.

Step 2: Sort the points based on distance.

Step 3: Select the first K points.

Step 4: Print the closest points.

Python Code:

main.py Visualize

```
points = [[1, 3], [-2, 2], [5, 8], [0, 1]]
k = 2
points.sort(key=lambda p: p[0]**2 + p[1]**2)
print(points[:k])
```

Output:

Output

```
[[0, 1], [-2, 2]]
```

Time Complexity – O(N log N) | Space Complexity – O(N)

Q10. Four Sum Count

Aim:

To count the number of tuples where:

A[i] + B[j] + C[k] + D[l] = 0

Algorithm:

Step 1: Calculate all sums of A and B.

Step 2: Store their frequencies in a dictionary.

Step 3: Calculate sums of C and D.

Step 4: Check whether the negative sum exists.

Step 5: Add the frequency to the result.

Python Code:

main.py

```python
from collections import Counter
A = [1, 2]
B = [-2, -1]
C = [-1, 2]
D = [0, 2]
count = Counter()
for a in A:
    for b in B:
        count[a + b] += 1

result = 0
for c in C:
    for d in D:
        result += count[-(c + d)]
print("Number of tuples =", result)
```

Output:

Output

```
Number of tuples = 2
```

Time Complexity – O(N²) | Space Complexity – O(N²)

Q11. Median of Medians Algorithm

Aim:

To find the K-th smallest element using the Median of Medians technique.

Algorithm:

Step 1: Divide the array into groups of five elements.

Step 2: Find the median of each group.

Step 3: Find the median of the medians.

Step 4: Use it as the pivot.

Step 5: Divide elements into smaller and greater groups.

Step 6: Recursively find the K-th smallest element.

Python Code:

main.py Visualize

```
def median_of_medians(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k - 1]
    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(group)[len(group)//2] for group in
    groups]
    pivot = median_of_medians(medians, (len(medians)+1)//2)
    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    if k <= len(low):
        return median_of_medians(low, k)
    elif k == len(low) + 1:
        return pivot
    else:
        return median_of_medians(high, k - len(low) - 1)
arr = [12, 3, 5, 7, 19]
k = 2
print(median_of_medians(arr, k))
```

Output:

Output

```
5
```

Time Complexity – O(N) | Space Complexity – O(N)

Q12. Find K-th Smallest Element

Aim:

To find the K-th smallest element in an unsorted array.

Algorithm:

Step 1: Accept the unsorted array.

Step 2: Sort the array.

Step 3: Select the element at position K-1.

Step 4: Print the K-th smallest element.

Python Code:

main.py

```
arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27]
k = 5
arr.sort()
print("K-th smallest element =", arr[k - 1])
```

Output:

Output

```
K-th smallest element = 21
```

Time Complexity – O(N log N) | Space Complexity – O(1)

Q13. Meet in the Middle – Closest Subset Sum

Aim:

To find a subset whose sum is closest to the target sum using the Meet in the Middle technique.

Algorithm:

Step 1: Divide the array into two halves.

Step 2: Generate all possible subset sums of both halves.

Step 3: Compare combinations of sums.

Step 4: Find the sum closest to the target.

Step 5: Print the closest sum.

Python Code:

main.py Visualize

```python
from itertools import combinations
arr = [45, 34, 4, 12, 5, 2]
target = 42
sums = []
for r in range(len(arr) + 1):
    for subset in combinations(arr, r):
        sums.append(sum(subset))
closest = min(sums, key=lambda x: abs(x - target))
print("Closest Sum =", closest)
```

Output:

Output

```
Closest Sum = 43
```

Time Complexity – O(2^(N/2)) | Space Complexity – O(2^(N/2))

Q14. Meet in the Middle – Exact Subset Sum

Aim:

To determine whether a subset exists with an exact target sum.

Algorithm:

Step 1: Divide the array into two halves.

Step 2: Generate all subset sums of both halves.

Step 3: Check whether two sums combine to form the target.

Step 4: Return True if found.

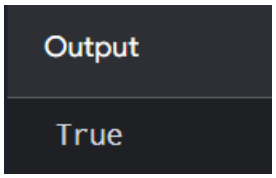
Step 5: Otherwise return False.

Python Code:

main.py Visualize

```python
def subset_sum(arr, target):
    n = len(arr)
    mid = n // 2
    left = arr[:mid]
    right = arr[mid:]
    left_sums = set()
    for mask in range(1 << len(left)):
        total = 0
        for i in range(len(left)):
            if mask & (1 << i):
                total += left[i]
        left_sums.add(total)
    for mask in range(1 << len(right)):
        total = 0
        for i in range(len(right)):
            if mask & (1 << i):
                total += right[i]
        if target - total in left_sums:
            return True
    return False
arr = [1, 3, 9, 2, 7, 12]
target = 15
print(subset_sum(arr, target))
```

Output:



Time Complexity – O(2^(N/2)) | Space Complexity – O(2^(N/2))

Q15. Strassen's Matrix Multiplication

Aim:

To multiply two 2 × 2 matrices using Strassen's Matrix Multiplication algorithm.

Algorithm:

Step 1: Take two 2 × 2 matrices.

Step 2: Calculate the seven Strassen products.

Step 3: Use the products to calculate C11, C12, C21, and C22.

Step 4: Form the resultant matrix.

Step 5: Print the result.

Python Code:

main.py

```
A = [[1, 7],
     [3, 5]]
B = [[6, 8],
     [4, 2]]
a, b = A[0]
c, d = A[1]
e, f = B[0]
g, h = B[1]
P1 = a * (f - h)
P2 = (a + b) * h
P3 = (c + d) * e
P4 = d * (g - e)
P5 = (a + d) * (e + h)
P6 = (b - d) * (g + h)
P7 = (a - c) * (e + f)

C11 = P5 + P4 - P2 + P6
C12 = P1 + P2
C21 = P3 + P4
C22 = P1 + P5 - P3 - P7

C = [[C11, C12],
     [C21, C22]]

print(C)
```

Output:

Output

```
[[34, 22], [38, 34]]
```

Time Complexity – O(N^2.81) | Space Complexity – O(N²)

Q16. Karatsuba Multiplication Algorithm

Aim:

To multiply two large integers using the Karatsuba algorithm.

Algorithm:

Step 1: Divide each number into two halves.

Step 2: Calculate the three required products.

Step 3: Calculate the middle product.

Step 4: Combine all values.

Step 5: Return the final multiplication result.

Python Code:

main.py Visualize

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y

    n = max(len(str(x)), len(str(y)))
    half = n // 2

    power = 10 ** half

    a = x // power
    b = x % power
    c = y // power
    d = y % power

    ac = karatsuba(a, c)
    bd = karatsuba(b, d)
    ad_bc = karatsuba(a + b, c + d) - ac - bd

    return ac * (10 ** (2 * half)) + ad_bc * power + bd

x = 1234
y = 5678

result = karatsuba(x, y)

print("Product =", result)
```

Output:

```
Output

Product = 7006652
```

Time Complexity – O(N^1.585) | Space Complexity – O(log N)